Operator ETL

Specification · Architecture · Build · Evaluation

Version 1.0 · August 2026
Local-first medallion pipeline: intake → warehouse → insights
Repository: /Users/Sour/operator-etl

Problem

- Silent bad data — invalid rows appear in dashboards without visibility
- No idempotency — re-running the same file double-counts records
- Tight coupling — each new source needs a new script
- Overconfident outputs — charts render when upstream data is broken
- No audit trail — operators can't answer what ran, when, or what failed

Goals & non-goals

Goals (v1)

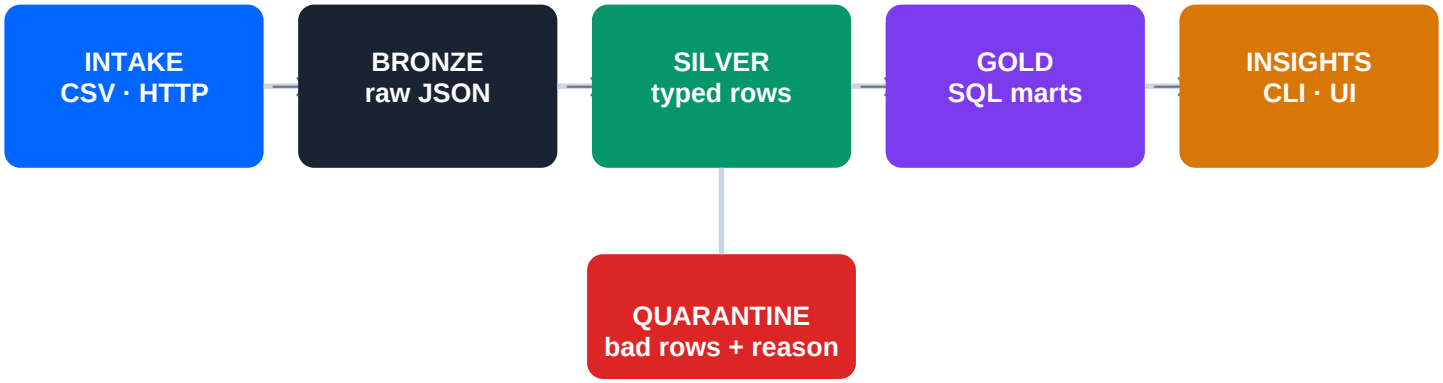
- Trustworthy intake — raw preserved, validate before silver
- Operator clarity — run logs, quarantine reasons
- Extensible sources via YAML registry
- Local-first — DuckDB on disk, no cloud required
- Deterministic ETL — Python + SQL, no LLM in critical path

Non-goals (v1)

- Cloud warehouse / hosted dashboard
- Airflow, Spark, Kafka, streaming
- LLM-generated insights
- Multi-tenant SaaS
- Domain adapters (Substack, finance) — later via registry

Specification — one sentence

- Drop a CSV or call an API → store raw data immutably in bronze
- Validate and type rows into silver; reject bad rows to quarantine
- Compute gold metrics in SQL; surface KPIs in CLI or dashboard
- Withhold insights when quality gates fail — fail closed, not optimistic
- New sources = one registry entry; pipeline runner stays the same



Quality gate: KPIs withheld if quarantine rate > 35% · freshness > 7 days · silver empty

Specification — layers

Layer	Responsibility	Technology
Extract	Read files/APIs, SHA-256 hash	Python, httpx
Bronze	Immutable raw JSON + metadata	DuckDB
Transform	Validate, type, dedupe	Pydantic v2
Quarantine	Isolate bad rows + error reason	DuckDB
Gold	Aggregated SQL marts	DuckDB SQL
Insights	KPIs when quality passes	CLI, Streamlit
Orchestration	Run logging, CLI commands	Typer

Source	Kind	Input	Purpose
demo	csv	samples/orders.csv	Demo & tests
inbox	csv_dir	drops/inbox/*.csv	Operator drop folder
http	http	GET JSON list	API intake stub

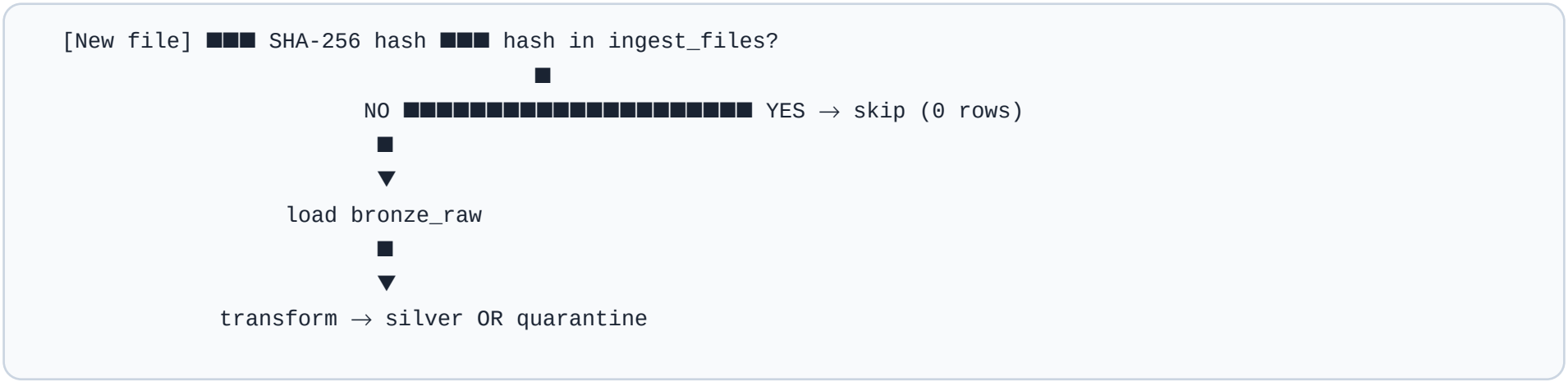
Specification — warehouse schema

Table / mart	Contents
ingest_files	File hash registry (idempotency)
pipeline_runs	Run audit: counts, status, timestamps
bronze_raw	Raw JSON + _source, _ingested_at, _row_num
silver_orders	Validated typed business rows
quarantine_orders	Rejected rows + error message
gold_kpis / gold_volume_daily / gold_top_skus	Aggregated metrics
gold_quality	Quarantine rate, freshness, row counts

Field	Type	Rule
order_id	string	Required, non-empty, unique
customer_id	string	Required
ordered_at	datetime	Parseable ISO timestamp
amount	float	Must be > 0
sku	string	Required
status	string	Required

Check	Default	On failure
Quarantine rate	≤ 35%	KPIs withheld
Freshness	≤ 7 days since ingest	KPIs withheld
Silver rows	> 0	KPIs withheld

Diagram — idempotency flow



How it looks — CLI

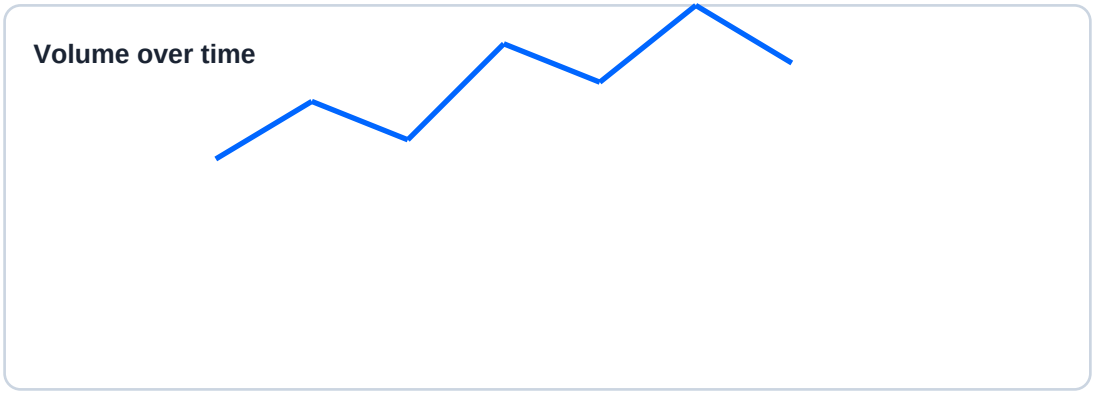
Terminal output after etl run --source demo

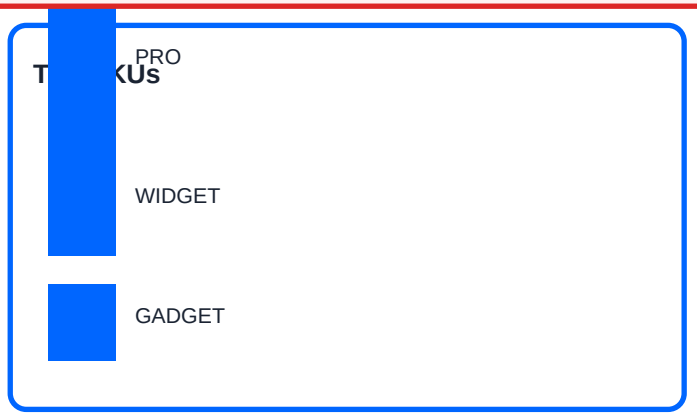
```
$ uv run etl run --source demo
run ok  source=demo  rows_in=21  silver=17  quarantined=4

Operator ETL insights
Quality: PASS   quarantine_rate=19.1%

KPIs
  orders       17
  customers    10
  revenue      1373.82
  avg order      80.81

Top SKUs
  SKU-PRO      orders=5  revenue=967.40
  SKU-WIDGET   orders=6  revenue=225.82
```

How it looks — dashboard

Orders	Customers	Revenue	Gate
17	10	1373.82	PASS

Quarantine (4 rows): empty order_id · invalid date · negative amount · non-numeric amount





Building process

1. **Scaffold**
   Repo · uv · DuckDB · CLI skeleton · source registry YAML
2. **Data plane**
   CSV/HTTP extract · bronze load · SHA-256 idempotency · run log
3. **Transform**
   Pydantic contract · silver load · quarantine table · dedupe
4. **Gold + insights**
   SQL marts · quality gate · CLI insight command
5. **Dashboard**
   Streamlit KPI cards · charts · quarantine expander · run history
6. **Evaluate**
   pytest: idempotency · quarantine · HTTP · quality gate block

Building process — repo layout

```
operator-etl/
  pipelines/demo.yaml     ← source registry
  samples/                ← demo CSV + HTTP JSON
  drops/inbox/            ← operator CSV drop
  sql/marts/              ← gold SQL
  src/operator_etl/       ← extract · load · transform · insights
  dashboard/app.py        ← Streamlit UI
  tests/                  ← pytest suite
  warehouse/operator.duckdb  (gitignored)
```

Evaluation — success criteria

#	Criterion	Status
1	CSV in drop folder runs end-to-end	Met
2	Invalid rows → quarantine	Met
3	KPIs: freshness, volume, SKU breakdown	Met
4	Re-ingest same file → 0 duplicate rows	Met
5	Quality gate blocks bad KPIs	Met
6	HTTP JSON source works	Met
7	Automated test coverage	9/9 pass

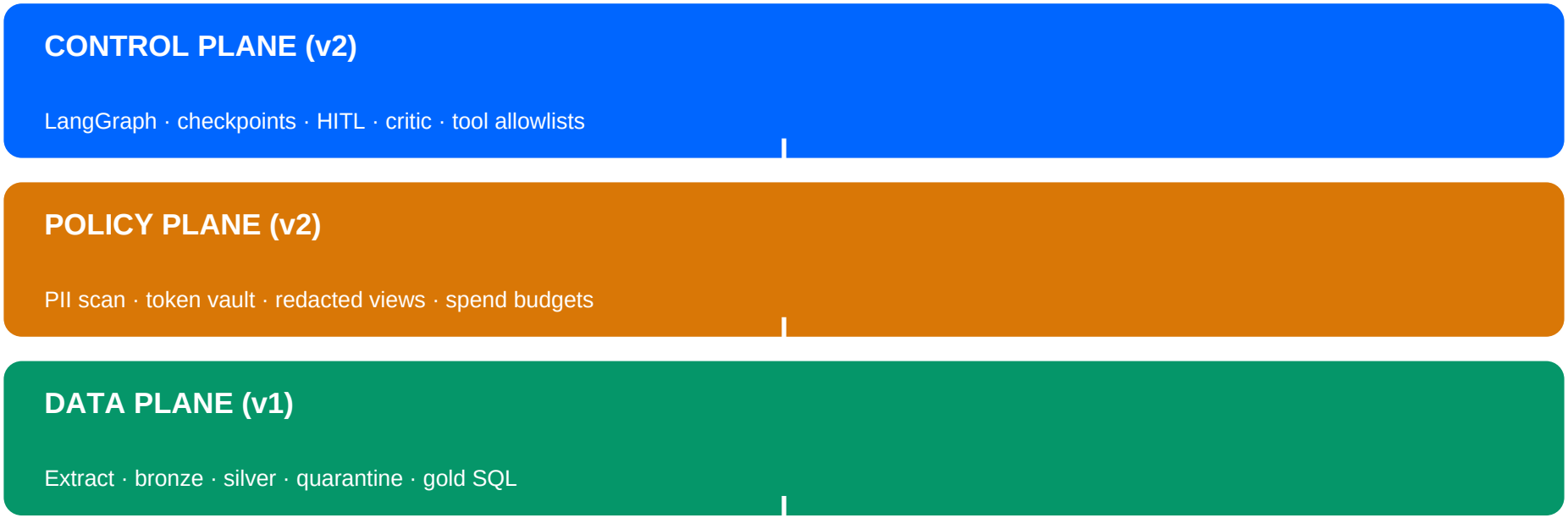
Evaluation — test matrix

Test	Validates	Result
test_ingest_is_idempotent	Hash skip on re-run	PASS
test_quarantine_invalid_rows	4 bad rows isolated	PASS
test_run_builds_gold	KPIs + gate pass	PASS
test_quality_gate_blocks	KPIs blocked > threshold	PASS
test_http_*	JSON extract + warehouse	PASS
test_source_registry	demo, inbox, http	PASS

Evaluation — demo run results

Metric	Value
Input rows (demo CSV)	21
Silver (valid)	17
Quarantined	4
Quarantine rate	19.1% → gate PASS
Re-run same file	0 new rows (hash skip)
Revenue (gold)	1373.82
Top SKU	SKU-PRO

Roadmap — agentic evolution (v2)



Roadmap

Phase	Capability
v1.0	Medallion ETL + CLI + dashboard (current)
v1.1	Real source adapters (Substack, finance CSV)
v1.2	Postgres/Supabase + scheduled runs
v2.0	LangGraph · PII gate · insight agent · critic · HITL
v2.1	Hosted dashboard · Langfuse observability

Recommendation

- Adopt v1 as the deterministic data plane — boring, testable, trustworthy
- Confirm canonical schema for first production data source
- Approve quality gate thresholds before live use
- Add v1.1 source adapter next; defer agentic layer until metrics are stable
- v2 agents orchestrate on top — they never replace ETL or bypass quarantine

Operator ETL v1

Intake → warehouse → insights

Questions?